

Lab # 01

Tasks

1. List all files in the current directory.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
```

2. List all files in human readable form.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ ls -lh
total 36K
drwxr-xr-x 2 saad-ali-162 saad-ali-162 4.0K May  8 18:55 Desktop
drwxr-xr-x 2 saad-ali-162 saad-ali-162 4.0K May  8 18:55 Documents
drwxr-xr-x 2 saad-ali-162 saad-ali-162 4.0K May  8 18:55 Downloads
drwxr-xr-x 2 saad-ali-162 saad-ali-162 4.0K May  8 18:55 Music
drwxr-xr-x 2 saad-ali-162 saad-ali-162 4.0K May  8 18:55 Pictures
drwxr-xr-x 2 saad-ali-162 saad-ali-162 4.0K May  8 18:55 Public
drwx----- 3 saad-ali-162 saad-ali-162 4.0K May  8 18:55 snap
drwxr-xr-x 2 saad-ali-162 saad-ali-162 4.0K May  8 18:55 Templates
drwxr-xr-x 2 saad-ali-162 saad-ali-162 4.0K May  8 18:55 Videos
```

3. Display your current directory.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ pwd
/home/saad-ali-162
```

4. Change to the / directory.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ cd /
saad-ali-162@saad-ali-162-VirtualBox:/$
```

5. Go to the ~ directory from the root directory.

```
saad-ali-162@saad-ali-162-VirtualBox:/$ cd ~/
saad-ali-162@saad-ali-162-VirtualBox:~$
```

6. List the contents of the root directory.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ ls /
bin          home         mnt          sbin.usr-is-merged  usr
bin.usr-is-merged  lib         opt          snap                var
boot         lib64        proc         srv
cdrom        lib.usr-is-merged  root         swap.img
dev          lost+found  run          sys
etc          media       sbin         tmp
```

7. List the files in /boot in a human readable format.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ ls -lh /boot
total 87M
-rw-r--r-- 1 root root 281K Apr 20 02:46 config-6.8.0-31-generic
drwxr-xr-x 5 root root 4.0K May 8 18:52 grub
lrwxrwxrwx 1 root root 27 May 8 18:51 initrd.img -> initrd.img-6.8.0-31-generic
-rw-r--r-- 1 root root 63M May 8 18:52 initrd.img-6.8.0-31-generic
lrwxrwxrwx 1 root root 27 May 8 18:51 initrd.img.old -> initrd.img-6.8.0-31-generic
-rw-r--r-- 1 root root 140K Apr 8 21:12 memtest86+ia32.bin
-rw-r--r-- 1 root root 141K Apr 8 21:12 memtest86+ia32.efi
-rw-r--r-- 1 root root 145K Apr 8 21:12 memtest86+x64.bin
-rw-r--r-- 1 root root 146K Apr 8 21:12 memtest86+x64.efi
-rw----- 1 root root 8.7M Apr 20 02:46 System.map-6.8.0-31-generic
lrwxrwxrwx 1 root root 24 May 8 18:51 vmlinuz -> vmlinuz-6.8.0-31-generic
-rw----- 1 root root 15M Apr 20 02:48 vmlinuz-6.8.0-31-generic
lrwxrwxrwx 1 root root 24 May 8 18:51 vmlinuz.old -> vmlinuz-6.8.0-31-generic
```

8. Create a directory testdir in your ~ directory.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ mkdir ~/testdir
saad-ali-162@saad-ali-162-VirtualBox:~$ ls
Desktop  Downloads  Pictures  snap      testdir
Documents Music      Public   Templates Videos
```

9. Create in one command the directories ~/dir1/dir2/dir3 (dir3 is a subdirectory from dir2, and dir2 is a subdirectory from dir1).

```
saad-ali-162@saad-ali-162-VirtualBox:~$ mkdir -p ~/dir1/dir2/dir3
saad-ali-162@saad-ali-162-VirtualBox:~$ ls
Desktop  Documents  Music      Public  Templates  Videos
dir1     Downloads  Pictures   snap    testdir
```

10. Remove the directory testdir.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ rm -r ~/testdir
saad-ali-162@saad-ali-162-VirtualBox:~$ ls
Desktop  Documents  Music      Public  Templates
dir1     Downloads  Pictures   snap    Videos
```

11. Move TestingDir from ~ to ~/Tests directories.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ mkdir ~/TestingDir
saad-ali-162@saad-ali-162-VirtualBox:~$ mkdir ~/Tests
saad-ali-162@saad-ali-162-VirtualBox:~$ ls
Desktop  Documents  Music      Public  Templates  Tests
dir1     Downloads  Pictures   snap    TestingDir  Videos
saad-ali-162@saad-ali-162-VirtualBox:~$ mv ~/TestingDir ~/Tests
saad-ali-162@saad-ali-162-VirtualBox:~$ ls
Desktop  Documents  Music      Public  Templates  Videos
dir1     Downloads  Pictures   snap    Tests
```

12. Rename a directory from TesingDir to testingDir

```
saad-ali-162@saad-ali-162-VirtualBox:~$ mv ~/Tests/TestingDir ~/Tests/testingDir
saad-ali-162@saad-ali-162-VirtualBox:~$ ls
Desktop  Documents  Music      Public  Templates  Videos
dir1     Downloads  Pictures   snap    Tests
saad-ali-162@saad-ali-162-VirtualBox:~$ cd ~/Tests
saad-ali-162@saad-ali-162-VirtualBox:~/Tests$ ls
testingDir
```

Lab # 02

Tasks:

1. Use ls to output the contents of the /etc/ directory to a file called etc.txt.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ ls /etc/ > etc.txt
saad-ali-162@saad-ali-162-VirtualBox:~$
```

2. Make a file bioData.txt and check that it contains your name using grep, also use i & v flags.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ cat > biodata.txt
Saad Ali
Section C
2022-BCS-162
saad-ali-162@saad-ali-162-VirtualBox:~$ grep -i saad < biodata.txt
Saad Ali
```

3. Append information in bioData.txt.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ cat biodata.txt
Saad Ali
Section C
2022-BCS-162
```

4. Sort the file number.txt using flags.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ sort -n number.txt
0
1
1
2
2
3
4
4
```

5. Show all the information like total lines, total words and total character inside of your file.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ wc biodata.txt
3  5 33 biodata.txt
```

6. Change case of the letters of your file bioData.txt.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ tr '[:lower:]' '[:upper:]' < biodata.txt
SAAD ALI
SECTION C
2022-BCS-162
```

7.Implement stderr (2> and 2>&1).

```
saad-ali-162@saad-ali-162-VirtualBox:~$ ls xyz.txt 2> error.txt
saad-ali-162@saad-ali-162-VirtualBox:~$ cat < error.txt
ls: cannot access 'xyz.txt': No such file or directory
saad-ali-162@saad-ali-162-VirtualBox:~$ ls xyz.txt > output.txt 2>&1
saad-ali-162@saad-ali-162-VirtualBox:~$ cat < output.txt
ls: cannot access 'xyz.txt': No such file or directory
```

8.Show last 5 data from numbers.txt

```
saad-ali-162@saad-ali-162-VirtualBox:~$ tail -n 5 number.txt
4
0
9
8
7
```

9.Count all duplicates in your file bioData.txt

```
saad-ali-162@saad-ali-162-VirtualBox:~$ sort biodata.txt | uniq -c
    1 2022-BCS-162
    1 Saad Ali
    1 Section C
```

10.Show your os version.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
```

11.Compress the file bioData.txt using gzip and bzip2 algorithm.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ gzip -c biodata.txt > biodata.txt.gz
saad-ali-162@saad-ali-162-VirtualBox:~$ bzip2 -k biodata.txt
```

12. Show contents of the file bioData.txt.gz without decompressing.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ zcat biodata.txt.gz
Saad Ali
Section C
2022-BCS-162
```

Lab # 03

Tasks:

1.Create two groups webProject and SQA.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ sudo groupadd Webproject
[sudo] password for saad-ali-162:
saad-ali-162@saad-ali-162-VirtualBox:~$ sudo groupadd SQA
```

2.Create two users , user1 and user2 and add them in a group webProject and SQA respectively.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ sudo useradd -G Webproject user1
saad-ali-162@saad-ali-162-VirtualBox:~$ sudo useradd -G SQA user2
```

3.Show All users.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ tail -5 /etc/passwd
nm-openvpn:x:121:122:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin
gnome-remote-desktop:x:985:985:GNOME Remote Desktop:/var/lib/gnome-remote-desktop:/usr/sbin/nologin
saad-ali-162:x:1000:1000:Saad Ali :/home/saad-ali-162:/bin/bash
user1:x:1001:1003:/:/home/user1:/bin/sh
user2:x:1002:1004:/:/home/user2:/bin/sh
```

4.Show All groups.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ tail -5 /etc/group
saad-ali-162:x:1000:
Webproject:x:1001:user1
SQA:x:1002:user2
user1:x:1003:
user2:x:1004:
```

5.Set user1 as group member of SQA.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ sudo usermod -aG SQA user1
saad-ali-162@saad-ali-162-VirtualBox:~$
```

6.Switch to user2.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ su - user2
```

7.Create a filename OSLabFile.txt and change its permission , read and execute only in group & user.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ touch OSLabFile.txt
saad-ali-162@saad-ali-162-VirtualBox:~$ chmod 550 OSLabFile.txt
```

8. Delete user1.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ sudo userdel -r user1
userdel: user1 mail spool (/var/mail/user1) not found
userdel: user1 home directory (/home/user1) not found
```

9. Delete SQA.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ sudo groupdel SQA
```

10. Change password of user1

```
saad-ali-162@saad-ali-162-VirtualBox:~$ sudo useradd user1
saad-ali-162@saad-ali-162-VirtualBox:~$ sudo passwd user1
New password:
Retype new password:
passwd: password updated successfully
```


Lab # 04

Tasks:

1.Create a MyInfo.txt using vi editor.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ vi Myinfo.txt
```

2.Write Your name , roll number and section in MyInfo.txt.

```
Saad Ali  
2022-BCS-162  
C
```

3.Save the File.

```
~  
:wq
```

4.Delete your section from MyInfo.txt.

```
Saad Ali  
2022-BCS-162  
|
```

5.Show content of MyInfo.txt using cat.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ cat Myinfo.txt  
Saad Ali  
2022-BCS-162
```

6.Print host name

```
saad-ali-162@saad-ali-162-VirtualBox:~$ hostname  
saad-ali-162-VirtualBox
```

7.Print your(system) user name.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ whoami  
saad-ali-162
```

8.Show the locatoin where ls ,pwd ,rmdir & cat source code is present.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ which ls  
/usr/bin/ls  
saad-ali-162@saad-ali-162-VirtualBox:~$ which pwd  
/usr/bin/pwd  
saad-ali-162@saad-ali-162-VirtualBox:~$ which rmdir  
/usr/bin/rmdir  
saad-ali-162@saad-ali-162-VirtualBox:~$ which cat  
/usr/bin/cat
```

```
saad-ali-162@saad-ali-162-VirtualBox:~$ gedit saad.c
```

```
saad-ali-162@saad-ali-162-VirtualBox:~$ chmod +x saad.c
```

```
saad-ali-162@saad-ali-162-VirtualBox:~$ gcc -o saad saad.c -lpthread
```

```
#include<unistd.h>
```

```
void* thread_function(void* args)
```

```
{  
    int i=0;  
    for(i=0;i<10;i++){  
        printf("%d\n",i);  
        sleep(1);  
    }  
}
```

```
int main(){  
    pthread_t id;  
    printf("Before thread is created\n");  
    int ret=pthread_create(&id,0,&thread_function,0);  
    pthread_join(id,0);  
    printf("After thread is executed :\n");  
    return 0;  
}
```

Lab # 05

Tasks:

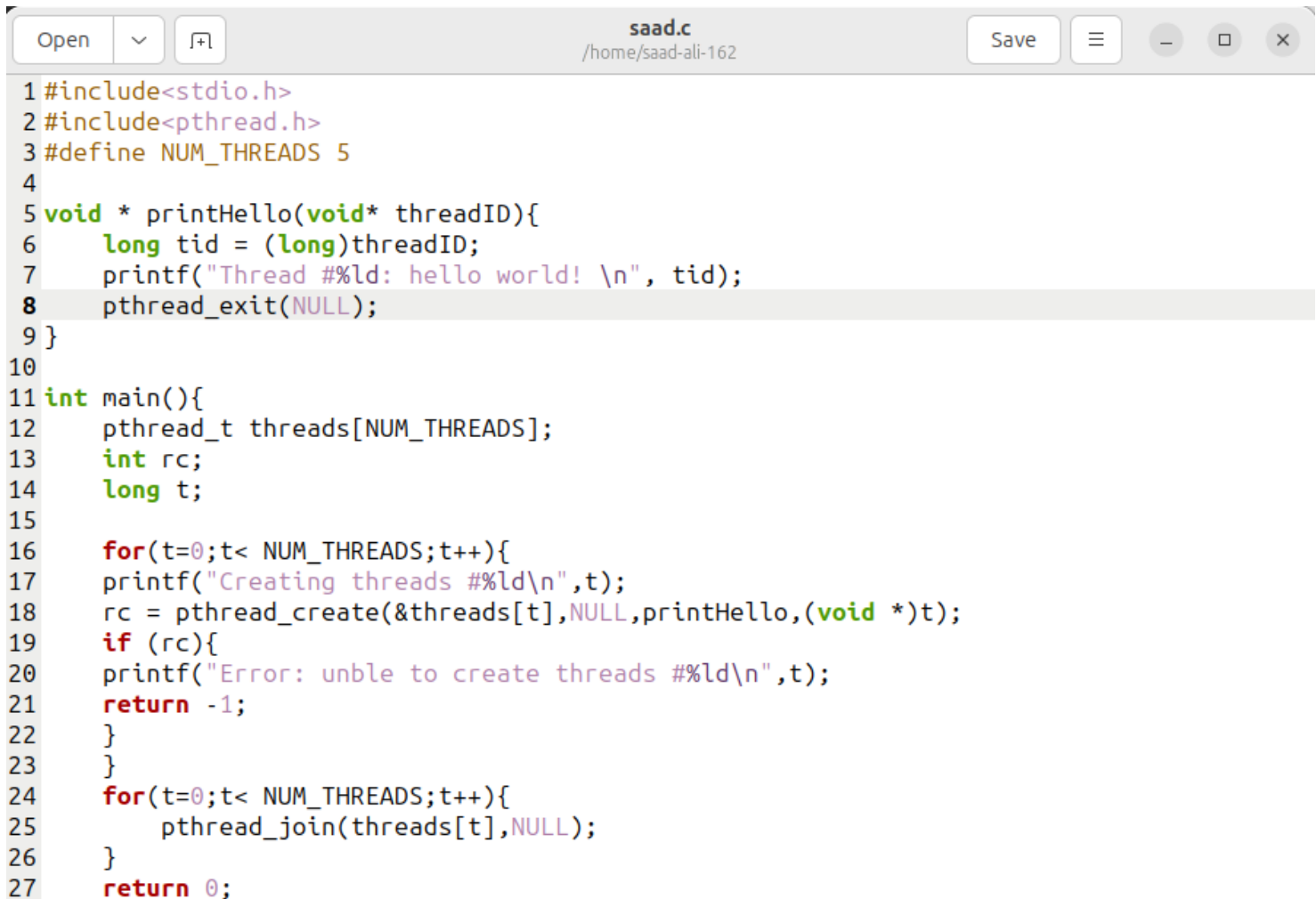
1. Create a file OSLabFile5.1 with date & time 25th January, 2025 3:00 PM.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ touch -t 2501251500 OSLabFile5.1
```

2. Find the binary and source file location of "ls" command

```
saad-ali-162@saad-ali-162-VirtualBox:~$ which ls
/usr/bin/ls
```

3. Creates 5 threads each thread prints a "Hello World!" message, and then terminates.



```
saad.c
/home/saad-ali-162

1 #include<stdio.h>
2 #include<pthread.h>
3 #define NUM_THREADS 5
4
5 void * printHello(void* threadID){
6     long tid = (long)threadID;
7     printf("Thread #%ld: hello world! \n", tid);
8     pthread_exit(NULL);
9 }
10
11 int main(){
12     pthread_t threads[NUM_THREADS];
13     int rc;
14     long t;
15
16     for(t=0;t< NUM_THREADS;t++){
17         printf("Creating threads #%ld\n",t);
18         rc = pthread_create(&threads[t],NULL,printHello,(void *)t);
19         if (rc){
20             printf("Error: unble to create threads #%ld\n",t);
21             return -1;
22         }
23     }
24     for(t=0;t< NUM_THREADS;t++){
25         pthread_join(threads[t],NULL);
26     }
27     return 0;
```

```
saad-ali-162@saad-ali-162-VirtualBox:~$ chmod +x saad.c
saad-ali-162@saad-ali-162-VirtualBox:~$ gcc -o saad saad.c -lpthread
saad-ali-162@saad-ali-162-VirtualBox:~$ ./saad
Creating threads #0
Creating threads #1
Creating threads #2
Thread #0: hello world!
Thread #1: hello world!
Creating threads #3
Creating threads #4
Thread #2: hello world!
Thread #3: hello world!
Thread #4: hello world!
```

Lab # 06

FCFS:

```

using System;
using System.Collections.Generic;
public class Process
{
    public int Id { get; set; }
    public int ArrivalTime { get; set; }
    public int BurstTime { get; set; }
    public int OriginalBurstTime { get; set; }
    public int WaitingTime { get; set; }
    public int TurnaroundTime { get; set; }
    public Process(int id, int arrivalTime, int burstTime)
    {
        Id = id;
        ArrivalTime = arrivalTime;
        BurstTime = burstTime;
        OriginalBurstTime = burstTime;
        WaitingTime = 0;
        TurnaroundTime = 0;
    }
}

public class FCFS
{
    public List<Process> Processes { get; set; }
    public FCFS(List<Process> processes)
    {
        Processes = processes;
    }
    public void Schedule()
    {
        int currentTime = 0;
        int n = Processes.Count;
        // Sort processes by arrival time (though FCFS doesn't strictly require this)
        Processes.Sort((a, b) => a.ArrivalTime.CompareTo(b.ArrivalTime));
        foreach (var proc in Processes)
        {
            // Process each job in arrival order
            if (proc.ArrivalTime > currentTime)
            {
                currentTime = proc.ArrivalTime;
            }
            proc.WaitingTime = currentTime - proc.ArrivalTime;
            proc.TurnaroundTime = proc.WaitingTime + proc.BurstTime;
            currentTime += proc.BurstTime;
        }
    }
    public void DisplayResults()
    {
        int totalWaitingTime = 0;
        int totalTurnaroundTime = 0;
        Console.WriteLine();
        Console.WriteLine("Process\tArrival\tBurst\tWaiting\tTurnaround");
        foreach (var proc in Processes)
        {
            Console.WriteLine($"{proc.Id}\t{proc.ArrivalTime}\t{proc.OriginalBurstTime}\t{proc.WaitingTime}\t{proc.TurnaroundTime}");
            totalWaitingTime += proc.WaitingTime;
            totalTurnaroundTime += proc.TurnaroundTime;
        }
    }
}

```

```

    }
    float averageWaitingTime = (float)totalWaitingTime / Processes.Count;
    float averageTurnaroundTime = (float)totalTurnaroundTime / Processes.Count;
    Console.WriteLine();
    Console.WriteLine($"Average Waiting Time: {averageWaitingTime:F2}");
    Console.WriteLine($"Average Turnaround Time: {averageTurnaroundTime:F2}");
}
}
public class Program
{
    public static void Main()
    {
        List<Process> processes = new List<Process>();
        Console.Write("Enter the number of processes: ");
        int numberOfProcesses = Convert.ToInt32(Console.ReadLine());
        for (int i = 1; i <= numberOfProcesses; i++)
        {
            Console.WriteLine();
            Console.Write($"Enter arrival time for process {i}: ");
            int arrivalTime = Convert.ToInt32(Console.ReadLine());
            Console.Write($"Enter burst time for process {i}: ");
            int burstTime = Convert.ToInt32(Console.ReadLine());
            processes.Add(new Process(i, arrivalTime, burstTime));
        }
        // Create an FCFS scheduler
        FCFS fcfs = new FCFS(processes);
        // Schedule the processes
        fcfs.Schedule();
        // Display the results
        fcfs.DisplayResults();
        Console.ReadLine();
    }
}

```

SJF:

```

using System;
using System.Collections.Generic;
public class Process
{
    public int Id { get; set; }
    public int ArrivalTime { get; set; }
    public int BurstTime { get; set; }
    public int OriginalBurstTime { get; set; }
    public int WaitingTime { get; set; }
    public int TurnaroundTime { get; set; }
    public Process(int id, int arrivalTime, int burstTime)
    {
        Id = id;
        ArrivalTime = arrivalTime;
        BurstTime = burstTime;
        OriginalBurstTime = burstTime;
        WaitingTime = 0;
        TurnaroundTime = 0;
    }
}
public class ShortestJobFirst
{
    public List<Process> Processes { get; set; }
    public ShortestJobFirst(List<Process> processes)
    {
        Processes = processes;
    }
    public void Schedule()

```

```

{
    // Sort processes based on arrival time, then burst time
    Processes.Sort((a, b) => {
        int compareArrival = a.ArrivalTime.CompareTo(b.ArrivalTime);
        if (compareArrival == 0)
        {
            return a.BurstTime.CompareTo(b.BurstTime);
        }
        return compareArrival;
    });
    int currentTime = 0;
    int completedProcesses = 0;
    int n = Processes.Count;
    while (completedProcesses < n)
    {
        // Find the next shortest job that has arrived
        Process shortestProcess = null;
        for (int i = 0; i < n; i++)
        {
            Process proc = Processes[i];
            if (proc.ArrivalTime <= currentTime && proc.BurstTime > 0)
            {
                if (shortestProcess == null || proc.BurstTime < shortestProcess.BurstTime)
                {
                    shortestProcess = proc;
                }
            }
        }
        if (shortestProcess != null)
        {
            // Process the shortest process found
            currentTime += shortestProcess.BurstTime;
            shortestProcess.TurnaroundTime = currentTime - shortestProcess.ArrivalTime;
            shortestProcess.WaitingTime = shortestProcess.TurnaroundTime -
            shortestProcess.OriginalBurstTime;
            // Mark the process as completed
            shortestProcess.BurstTime = 0;
            completedProcesses++;
        }
        else
        {
            // If no process found to execute, just increment the time
            currentTime++;
        }
    }
}

public void DisplayResults()
{
    int totalWaitingTime = 0;
    int totalTurnaroundTime = 0;
    Console.WriteLine();
    Console.WriteLine("Process\tArrival\tBurst\tWaiting\tTurnaround");
    foreach (var proc in Processes)
    {
        Console.WriteLine($"{proc.Id}\t{proc.ArrivalTime}\t{proc.OriginalBurstTime}\t{proc.WaitingTime}\t{proc.TurnaroundTime}");
        totalWaitingTime += proc.WaitingTime;
        totalTurnaroundTime += proc.TurnaroundTime;
    }
    float averageWaitingTime = (float)totalWaitingTime / Processes.Count;
    float averageTurnaroundTime = (float)totalTurnaroundTime / Processes.Count;
}

```

```

        Console.WriteLine();
        Console.WriteLine($"Average Waiting Time: {averageWaitingTime:F2}");
        Console.WriteLine($"Average Turnaround Time: {averageTurnaroundTime:F2}");
    }
}

public class Program
{
    public static void Main()
    {
        List<Process> processes = new List<Process>();
        Console.Write("Enter the number of processes: ");
        int numberOfProcesses = Convert.ToInt32(Console.ReadLine());
        for (int i = 1; i <= numberOfProcesses; i++)
        {
            Console.WriteLine();
            Console.Write($"Enter arrival time for process {i}: ");
            int arrivalTime = Convert.ToInt32(Console.ReadLine());
            Console.Write($"Enter burst time for process {i}: ");
            int burstTime = Convert.ToInt32(Console.ReadLine());
            processes.Add(new Process(i, arrivalTime, burstTime));
        }
        // Create a SJF scheduler
        ShortestJobFirst sjf = new ShortestJobFirst(processes);
        // Schedule the processes
        sjf.Schedule();
        // Display the results
        sjf.DisplayResults();
        Console.ReadLine();
    }
}

```

1. For below processes, write FCFS and SJF processes to calculate waiting time for each process and average waiting time.

Process	Arrival Time	Execute Time
P0	0	5
P1	1	3
P2	2	8
P3	3	6

FCFS:

```

Enter the number of processes: 4

Enter arrival time for process 1: 0
Enter burst time for process 1: 5

Enter arrival time for process 2: 1
Enter burst time for process 2: 3

Enter arrival time for process 3: 2
Enter burst time for process 3: 8

Enter arrival time for process 4: 3
Enter burst time for process 4: 6

Process Arrival Burst    Waiting Turnaround
1         0         5         0         5
2         1         3         4         7
3         2         8         6        14
4         3         6        13        19

Average Waiting Time: 5.75
Average Turnaround Time: 11.25

```


SJF:

```

Enter the number of processes: 4

Enter arrival time for process 1: 0
Enter burst time for process 1: 5

Enter arrival time for process 2: 1
Enter burst time for process 2: 3

Enter arrival time for process 3: 2
Enter burst time for process 3: 8

Enter arrival time for process 4: 3
Enter burst time for process 4: 6

Process Arrival Burst   Waiting Turnaround
1         0       5       0         5
2         1       3       4         7
3         2       8      12        20
4         3       6       5        11

Average Waiting Time: 5.25
Average Turnaround Time: 10.75

```

2. For below processes, write FCFS and SJF processes to calculate waiting time for each process and average waiting time.

Process	Arrival Time	Execute Time
P0	2	6
P1	5	2
P2	1	8
P3	0	3
P4	4	4

FCFS:

```

C:\Users\Saad Ali\source\repos\ConsoleApp2\ConsoleApp2\b
Enter the number of processes: 5

Enter arrival time for process 1: 2
Enter burst time for process 1: 6

Enter arrival time for process 2: 5
Enter burst time for process 2: 2

Enter arrival time for process 3: 1
Enter burst time for process 3: 8

Enter arrival time for process 4: 0
Enter burst time for process 4: 3

Enter arrival time for process 5: 4
Enter burst time for process 5: 4

Process Arrival Burst   Waiting Turnaround
4         0       3       0         3
3         1       8       2        10
1         2       6       9        15
5         4       4      13        17
2         5       2      16        18

Average Waiting Time: 8.00
Average Turnaround Time: 12.60

```

SJF:

```
C:\Users\Saad Ali\source\repos\ConsoleApp3\ConsoleApp3\bin\
Enter the number of processes: 5
Enter arrival time for process 1: 2
Enter burst time for process 1: 6
Enter arrival time for process 2: 5
Enter burst time for process 2: 2
Enter arrival time for process 3: 1
Enter burst time for process 3: 8
Enter arrival time for process 4: 0
Enter burst time for process 4: 3
Enter arrival time for process 5: 4
Enter burst time for process 5: 4

Process Arrival Burst   Waiting Turnaround
4          0         3         0         3
3          1         8        14        22
1          2         6         1         7
5          4         4         7        11
2          5         2         4         6

Average Waiting Time: 5.20
Average Turnaround Time: 9.80
```

Lab # 07

ROUND ROBIN:

```
using System;
class GFG
{
    public static void roundRobin(String[] p, int[] a, int[] b, int n)
    {
        // Result of average times
        int res = 0;
        int resc = 0;
        // For sequence storage
        String seq = "";
        // Copy the burst array and arrival array for not affecting the actual array
        int[] res_b = new int[b.Length];
        int[] res_a = new int[a.Length];
        for (int i = 0; i < res_b.Length; i++)
        {
            res_b[i] = b[i];
            res_a[i] = a[i];
        }
        // Critical time of system
        int t = 0;
        // For store the waiting time
        int[] w = new int[p.Length];
        // For store the Completion time
        int[] comp = new int[p.Length];
        while (true)
        {
            bool flag = true;
            for (int i = 0; i < p.Length; i++)
            {
                // These conditions are for if arrival is not on zero
                // Check that if they come before quantum time
                if (res_a[i] <= t)
                {
                    if (res_a[i] <= n)
                    {
                        if (res_b[i] > 0)
                        {
                            flag = false;
                            if (res_b[i] > n)
                            {
                                // Make decrease the burst time
                                t = t + n;
                                res_b[i] = res_b[i] - n;
                                res_a[i] = res_a[i] + n;
                                seq += "->" + p[i];
                            }
                        }
                        else
                        {
                            // For last time
                            t = t + res_b[i];
                            // Store completion time
                            comp[i] = t - a[i];
                            // Store waiting time
                            w[i] = t - b[i] - a[i];
                            res_b[i] = 0;
                            // Add sequence
                            seq += "->" + p[i];
                        }
                    }
                }
            }
        }
    }
}
```

```

    }
    else if (res_a[i] > n)
    {
        // If any process has less arrival time than the current process, execute
        for (int j = 0; j < p.Length; j++)
        {
            // Compare
            if (res_a[j] < res_a[i])
            {
                if (res_b[j] > 0)
                {
                    flag = false;
                    if (res_b[j] > n)
                    {
                        t = t + n;
                        res_b[j] = res_b[j] - n;
                        res_a[j] = res_a[j] + n;
                        seq += "->" + p[j];
                    }
                    else
                    {
                        t = t + res_b[j];
                        comp[j] = t - a[j];
                        w[j] = t - b[j] - a[j];
                        res_b[j] = 0;
                        seq += "->" + p[j];
                    }
                }
            }
        }
        // Now the previous process according to ith process
        if (res_b[i] > 0)
        {
            flag = false;
            // Check for greater burst times
            if (res_b[i] > n)
            {
                t = t + n;
                res_b[i] = res_b[i] - n;
                res_a[i] = res_a[i] + n;
                seq += "->" + p[i];
            }
            else
            {
                t = t + res_b[i];
                comp[i] = t - a[i];
                w[i] = t - b[i] - a[i];
                res_b[i] = 0;
                seq += "->" + p[i];
            }
        }
    }
    else if (res_a[i] > t)
    {
        t++;
        i--;
    }
}
// Exit the while loop
if (flag)
{

```

```

        break;
    }
}
Console.WriteLine();
Console.WriteLine("process\ttctime\twtime");
for (int i = 0; i < p.Length; i++)
{
    Console.WriteLine(" " + p[i] + "\t" + comp[i] + "\t" + w[i]);
    res = res + w[i];
    resc = resc + comp[i];
}
Console.WriteLine();
Console.WriteLine("Average waiting time is " + (float)res / p.Length);
Console.WriteLine("Average completion time is " + (float)resc / p.Length);
Console.WriteLine("Sequence is like that " + seq);
}
// Driver Code
public static void Main(String[] args)
{
    // Get the number of processes from the user
    Console.Write("Enter the number of processes: ");
    int numProcesses = int.Parse(Console.ReadLine());
    // Name of the processes
    String[] name = new String[numProcesses];
    for (int i = 0; i < numProcesses; i++)
    {
        name[i] = "p" + (i + 1);
    }
    Console.WriteLine();
    // Arrival time for every process
    int[] arrivaltime = new int[numProcesses];
    Console.WriteLine("Enter the arrival times:");
    for (int i = 0; i < numProcesses; i++)
    {
        Console.Write("Arrival time for " + name[i] + ": ");
        arrivaltime[i] = int.Parse(Console.ReadLine());
    }
    Console.WriteLine();
    // Burst time for every process
    int[] bursttime = new int[numProcesses];
    Console.WriteLine("Enter the burst times:");
    for (int i = 0; i < numProcesses; i++)
    {
        Console.Write("Burst time for " + name[i] + ": ");
        bursttime[i] = int.Parse(Console.ReadLine());
    }
    Console.WriteLine();
    // Quantum time of each process
    Console.Write("Enter the quantum time: ");
    int q = int.Parse(Console.ReadLine());
    // Call the function for output
    roundRobin(name, arrivaltime, bursttime, q);
    Console.ReadLine();
}
}

```

PRIORITY BASED:

```

using System;
class Program
{
    static int totalprocess;
    static int[][] proc;
    static int[] arrivaltime;
    static int[] bursttime;
    static int[] priority;
    // Driver code
    static void Main(string[] args)
    {
        // Get the number of processes from the user
        Console.Write("Enter the number of processes: ");
        totalprocess = int.Parse(Console.ReadLine());
        proc = new int[totalprocess][];
        arrivaltime = new int[totalprocess];
        bursttime = new int[totalprocess];
        priority = new int[totalprocess];
        // Get arrival time, burst time, and priority for each process
        for (int i = 0; i < totalprocess; i++)
        {
            Console.WriteLine();
            Console.Write($"Enter arrival time for process {i + 1}: ");
            arrivaltime[i] = int.Parse(Console.ReadLine());
            Console.Write($"Enter burst time for process {i + 1}: ");
            bursttime[i] = int.Parse(Console.ReadLine());
            Console.Write($"Enter priority for process {i + 1}: ");
            priority[i] = int.Parse(Console.ReadLine());
        }
        // Initialize the proc array
        for (int i = 0; i < totalprocess; i++)
        {
            proc[i] = new int[4];
            proc[i][0] = arrivaltime[i];
            proc[i][1] = bursttime[i];
            proc[i][2] = priority[i];
            proc[i][3] = i + 1;
        }
        // Sort by priority and then by arrival time
        Array.Sort(proc, (x, y) => x[2].CompareTo(y[2]));
        Array.Sort(proc, (x, y) => x[0].CompareTo(y[0]));
        Findgc();
        Console.ReadLine();
    }
    // Using FCFS Algorithm to find Waiting time
    static void GetWtTime(int[] wt)
    {
        // Declaring service array that stores cumulative burst time
        int[] service = new int[totalprocess];
        // Initialising initial elements of the arrays
        service[0] = 0;
        wt[0] = 0;
        for (int i = 1; i < totalprocess; i++)
        {
            service[i] = proc[i - 1][1] + service[i - 1];
            wt[i] = service[i] - proc[i][0] + 1;
            // If waiting time is negative, change it to zero
            if (wt[i] < 0)
            {
                wt[i] = 0;
            }
        }
    }
}

```

```

    }
}
// Filling turnaround time array
static void GetTatTime(int[] tat, int[] wt)
{
    for (int i = 0; i < totalprocess; i++)
    {
        tat[i] = proc[i][1] + wt[i];
    }
}
static void Findgc()
{
    // Declare waiting time and turnaround time array
    int[] wt = new int[totalprocess];
    int[] tat = new int[totalprocess];
    int wavg = 0;
    int tavg = 0;
    // Function call to find waiting time array
    GetWtTime(wt);
    // Function call to find turnaround time
    GetTatTime(tat, wt);
    int[] stime = new int[totalprocess];
    int[] ctime = new int[totalprocess];
    stime[0] = 1;
    ctime[0] = stime[0] + tat[0];
    Console.WriteLine();

    Console.WriteLine("Process_no\tStart_time\tComplete_time\tTurn_Around_Time\tWaiting_Time");
    // Calculating starting and ending time
    for (int i = 0; i < totalprocess; i++)
    {
        wavg += wt[i];
        tavg += tat[i];
        Console.WriteLine(proc[i][3] + "\t\t" + stime[i] + "\t\t" + ctime[i] + "\t\t" +
tat[i]
        + "\t\t\t" + wt[i]);
        // Display the process details
        if (i != totalprocess - 1)
        {
            stime[i + 1] = ctime[i];
            ctime[i + 1] = stime[i + 1] + tat[i + 1] - wt[i + 1];
        }
    }
    Console.WriteLine();
    // Display the average waiting time and average turn around time
    Console.WriteLine("Average waiting time is: " + (double)wavg / totalprocess);
    Console.WriteLine("Average turnaround time is: " + (double)tavg / totalprocess);
}
}

```

1. Perform the above code with 8 Number of processes by randomly setting the arrival time and burst time and priority.

(a) Use Round Robin Algorithm

(b) Use Priority Based Algorithm

```

C:\Users\Saad Ali\source\repos\ConsoleApp2\ConsoleApp2\b...
Burst time for p2: 8
Burst time for p3: 12
Burst time for p4: 14
Burst time for p5: 16
Burst time for p6: 18
Burst time for p7: 22
Burst time for p8: 24

Enter the quantum time: 4

process ctime  wtime
p1      4       0
p2      27      19
p3      54      42
p4      83      69
p5      86      70
p6      99      81
p7      108     86
p8      111     87

Average waiting time is 56.75
Average completion time is 71.5
Sequence is like that ->p1->p2->p3->p4->p5->p6->p2->p7->p3->p
8->p4->p5->p6->p3->p7->p4->p8->p5->p6->p7->p8->p4->p5->p6->p7
->p8->p6->p7->p8->p7->p8

```

```

C:\Users\Saad Ali\source\repos\ConsoleApp3\ConsoleApp3\bin\Debug\ConsoleApp3.exe
Enter arrival time for process 6: 9
Enter burst time for process 6: 8
Enter priority for process 6: 11

Enter arrival time for process 7: 9
Enter burst time for process 7: 8
Enter priority for process 7: 7

Enter arrival time for process 8: 6
Enter burst time for process 8: 5
Enter priority for process 8: 4

Process_no    Start_time    Complete_time    Turn_Around_Time    Waiting_Time
1             1             5                4                   0
4             5             9                6                   2
2             9             14               10                  5
8             14            19               13                  8
3             19            25               18                  12
5             25            32               24                  17
7             32            40               31                  23
6             40            48               39                  31

Average waiting time is: 12.25
Average turnaround time is: 18.125

```


2. Perform the above code with 10 Number of processes by randomly setting the arrival time and burst time and priority.

(a) Use Round Robin Algorithm

(b) Use Priority Based Algorithm

```
C:\Users\Saad Ali\source\repos\ConsoleApp2\ConsoleApp2\bin\Debug\ConsoleApp2....
Arrival time for p3: 3
Arrival time for p4: 4
Arrival time for p5: 5
Arrival time for p6: 6
Arrival time for p7: 7
Arrival time for p8: 8
Arrival time for p9: 9
Arrival time for p10: 10

Enter the burst times:
Burst time for p1: 2
Burst time for p2: 4
Burst time for p3: 6
Burst time for p4: 8
Burst time for p5: 10
Burst time for p6: 12
Burst time for p7: 14
Burst time for p8: 16
Burst time for p9: 18
Burst time for p10: 20

Enter the quantum time: 5

process ctime  wtime
p1      2       0
p2      5       1
p3      35      29
p4      42      34
p5      51      41
p6      77      65
p7      80      66
p8      95      79
p9      97      79
p10     101     81

Average waiting time is 47.5
Average completion time is 58.5
Sequence is like that ->p1->p2->p3->p4->p5->p6->p7->p8->p3->p9->p4->p10->p5->
```

```
C:\Users\Saad Ali\source\repos\ConsoleApp3\ConsoleApp3\bin\Debug\ConsoleApp3.exe
Enter burst time for process 7: 2
Enter priority for process 7: 1

Enter arrival time for process 8: 9
Enter burst time for process 8: 8
Enter priority for process 8: 7

Enter arrival time for process 9: 6
Enter burst time for process 9: 5
Enter priority for process 9: 4

Enter arrival time for process 10: 3
Enter burst time for process 10: 2
Enter priority for process 10: 1

Process_no      Start_time      Complete_time    Turn_Around_Time    Waiting_Time
1               1               5                4                   0
4               5               7                4                   2
7               7               9                6                   4
10              9               11               8                   6
3               11              16               10                  5
6               16              21               15                  10
9               21              26               20                  15
2               26              34               25                  17
5               34              42               33                  25
8               42              50               41                  33

Average waiting time is: 11.7
Average turnaround time is: 16.6
```

Lab # 10

1. Take two inputs from user and then perform arithmetic operations

```
#!/bin/bash

echo -n "Enter first number: "
read num1

echo -n "Enter second number:"
read num2

sum=$(expr $num1 + $num2)

difference=$(expr $num1 - $num2)
product=$(expr $num1 \* $num2)
quotient=$(expr $num1 / $num2)
remainder=$(expr $num1 % $num2)

echo "Sum: $sum"
echo "Difference: $difference"
echo "Product: $product"
echo "Quotient: $quotient"
echo "Remainder: $remainder"
~
~

saad-ali-162@saad-ali-162-VirtualBox:~$ ./lab10.sh
Enter first number: 20
Enter second number:10
Sum: 30
Difference: 10
Product: 200
Quotient: 2
Remainder: 0
```

2. Use system variables and user defined variables to show how that value can be accessed.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ vi lab10-2
```

```
#!/bin/bash

echo "System Variables"
echo "User: $USER"
echo "Home Directory: $HOME"
echo "Shell: $SHELL"
echo "Hostname: $HOSTNAME"
echo ""

echo "User-Defined Variables"
var1="Hello World!"
echo "var1: $var1"

num1=10
num2=5
sum=$((num1+num2))
echo "Sum of $num1 and $num2 is: $sum"
echo ""

echo "Current Date and Time:"
current_datetime=$(date)
echo "$current_datetime"
```

```
saad-ali-162@saad-ali-162-VirtualBox:~$ chmod 755 lab10-2
saad-ali-162@saad-ali-162-VirtualBox:~$ ./lab10-2
```

```
System Variables
User: faisal152
Home Directory: /home/faisal152
Shell: /bin/bash
Hostname: faisal152

User-Defined Variables
var1: Hello World!
Sum of 10 and 5 is: 15

Current Date and Time:
Wed 26 Jun 2024 06:46:42 AM GMT
```

3.Show results of double , single and back quote.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ vi lab10-3.sh
```

```
#!/bin/bash

var="Hello World!"
echo "Double quotes: $var"

var='Hello World!'
echo 'Single quotes: $var'

current_date=`date`
echo "Backticks (date command): $current_date"
```

```
saad-ali-162@saad-ali-162-VirtualBox:~$ chmod 755 lab10-3.sh
saad-ali-162@saad-ali-162-VirtualBox:~$ ./lab10-3.sh
Double quotes: Hello World!
Single quotes: $var
Backticks (date command): Wed 26 Jun 2024 07:03:26 AM GMT
```

Lab # 11

1. Write a program to generate the following output by using while loop

The value of x is 10
The value of x is 9
The value of x is 8
The value of x is 7
The value of x is 6
The value of x is 5
The value of x is 4
The value of x is 3
The value of x is 2
The value of x is 1

```
saad-ali-162@saad-ali-162-VirtualBox:~$ vi lab11-1.sh
```

```
#!/bin/bash

x=10

while [$x -gt 0]; do
    echo "The value of x is $x"
    x=$((expr $x - 1))
done
```

```
saad-ali-162@saad-ali-162-VirtualBox:~$ chmod 755 lab11-1.sh
saad-ali-162@saad-ali-162-VirtualBox:~$ ./lab11-1.sh
```

```
The value of x is 10
The value of x is 9
The value of x is 8
The value of x is 7
The value of x is 6
The value of x is 5
The value of x is 4
The value of x is 3
The value of x is 2
The value of x is 1
```

2. Write a program to generate the table of any number by using for loop.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ vi lab11-2.sh
```

```
#!/bin/bash

echo -n "Enter a number to generate its multiplication table: "
read num

echo "Multiplication table of $num:"
for (( i=1; i<=10; i++ )); do
    result=$((expr $num \* $i))
    echo "$num * $i = $result"
done
```

```
saad-ali-162@saad-ali-162-VirtualBox:~$ chmod 755 lab11-2.sh
saad-ali-162@saad-ali-162-VirtualBox:~$ ./lab11-2.sh
Enter a number to generate its multiplication table:
Multiplication table of 3:
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
```

3. Write a program to generate the factorial of a number.

```
saad-ali-162@saad-ali-162-VirtualBox:~$ vi lab11-3.sh
#!/bin/bash

factorial() {
    num=$1
    fact=1

    for (( i = 1; i <= num; i++ )); do
        fact=$((expr $fact \* $i))
    done

    echo $fact
}

echo -n "Enter a number find its factorial: "
read num

if [[ $num =~ ^[0-9]+$ ]]; then
    result=$(factorial $num)
    echo "Factorial of $num is: $result"
else
    echo "Error: Please enter a valid non negative integer."
fi
```

```
saad-ali-162@saad-ali-162-VirtualBox:~$ chmod 755 lab11-3.sh
saad-ali-162@saad-ali-162-VirtualBox:~$ ./lab11-3.sh
```

```
Enter a number find its factorial: 3
Factorial of 3 is: 6
```

4. Write a program to show the grades and % of students (with students details).

```
saad-ali-162@saad-ali-162-VirtualBox:~$ vi lab11-4.sh
```

```
#!/bin/bash

calculate_percentage(){
    total_marks=$1
    max_marks=$2

    percentage=$(echo "scale=2; ($total_marks / $max_marks) * 100" | bc)
    echo $percentage
}

calculate_grade(){
    percentage=$1

    if (( $(echo "$percentage >= 90" | bc -l) )); then
        echo "A"
    elif (( $(echo "$percentage >= 80" | bc -l) )); then
        echo "B"
    elif (( $(echo "$percentage >= 70" | bc -l) )); then
        echo "C"
    elif (( $(echo "$percentage >= 60" | bc -l) )); then
        echo "D"
    else
        echo "F"
    fi
}

display_student_details(){
    name=$1
    roll_number=$2
    marks=$3
    max_marks=$4

    percentage=$(calculate_percentage $marks $max_marks)
    grade=$(calculate_grade $percentage)

    echo "Student Name: $name"
    echo "Roll number: $roll_number"
    echo "Marks Obtained: $marks out of $max_marks"
    echo "Percentage: $percentage%"
    echo "Grade: $grade"
    echo
}

students=(
    "Faisal|001|450|500"
    "Farhan|002|400|500"
    "Furqan|003|350|500"
)

for student in "${students[@]"; do
    IFS='|' read name roll_number marks max_marks <<< "$student"
    display_student_details "$name" "$roll_number" "$marks" "$max_marks"
done
```



```
saad-ali-162@saad-ali-162-VirtualBox:~$ chmod 755 lab11-4.sh  
saad-ali-162@saad-ali-162-VirtualBox:~$ ./lab11-4.sh
```

```
Student Name: Faisal  
Roll number: 001  
Marks Obtained: 450 out of 500  
Percentage: 90.00%  
Grade: A
```

```
Student Name: Farhan  
Roll number: 002  
Marks Obtained: 400 out of 500  
Percentage: 80.00%  
Grade: B
```

```
Student Name: Furqan  
Roll number: 003  
Marks Obtained: 350 out of 500  
Percentage: 70.00%  
Grade: C
```

Lab # 13a

1. Write a Bash script that takes a username as an argument and prints the user's ID, home directory, and login shell using switch case.

```
#!/bin/bash

# Check if a username is provided as an argument
if [ -z "$1" ]; then
    echo "Usage: $0 <username>"
    exit 1
fi

username=$1

# Get user details
user_id=$(id -u "$username" 2>/dev/null)
home_directory=$(getent passwd "$username" | cut -d: -f6)
login_shell=$(getent passwd "$username" | cut -d: -f7)

# Check if the user exists
if [ -z "$user_id" ]; then
    echo "User '$username' does not exist."
    exit 1
fi

# Switch case to print user details
case $1 in
    $username)
        echo "User ID: $user_id"
        echo "Home Directory: $home_directory"
        echo "Login Shell: $login_shell"
        ;;
    *)
        echo "No information found for user '$username'."
        ;;
esac
```

```
saad-ali-162@saad-ali-162-VirtualBox:~$ nano lab13_1.sh
saad-ali-162@saad-ali-162-VirtualBox:~$ chmod +x lab13_1.sh
saad-ali-162@saad-ali-162-VirtualBox:~$ ./lab13_1.sh
Usage: ./lab13_1.sh <username>
saad-ali-162@saad-ali-162-VirtualBox:~$ ./lab13_1.sh saad
User 'saad' does not exist.
```

2. Write a Bash script that takes three arguments: two numbers and an operator (+, -, *, /). The script should perform the calculation and print the result handle by using switch case .

```
#!/bin/bash

# Check if exactly three arguments are provided
if [ "$#" -ne 3 ]; then
    echo "Usage: $0 <number1> <operator> <number2>"
    exit 1
fi

num1=$1
operator=$2
num2=$3

# Switch case to perform the calculation
case $operator in
    +)
        result=$(echo "$num1 + $num2" | bc)
        echo "Result: $result"
        ;;
    -)
        result=$(echo "$num1 - $num2" | bc)
        echo "Result: $result"
        ;;
    \*)
        result=$(echo "$num1 * $num2" | bc)
        echo "Result: $result"
        ;;
    /)
        if [ "$num2" -eq 0 ]; then
            echo "Error: Division by zero is not allowed."
            exit 1
        fi
        result=$(echo "scale=2; $num1 / $num2" | bc)
        echo "Result: $result"
        ;;
    *)
        echo "Invalid operator. Use +, -, *, or /."
        exit 1
        ;;
esac

saad-ali-162@saad-ali-162-VirtualBox:~$ chmod +x lab13_2.sh
saad-ali-162@saad-ali-162-VirtualBox:~$ ./lab13_2.sh 3 + 9
Result: 12
```

3. Write a Bash script that counts and prints the number of files in a given directory and its subdirectories handle by using switch case.

```
#!/bin/bash

# Function to count files
count_files() {
    directory=$1
    file_count=$(find "$directory" -type f | wc -l)
    echo "Total number of files in '$directory' and its subdirectories: $file_count"
}

# Check if a directory is provided as an argument
if [ -z "$1" ]; then
    echo "Usage: $0 <directory>"
    exit 1
fi

directory=$1

# Check if the provided argument is a valid directory
if [ ! -d "$directory" ]; then
    echo "'$directory' is not a valid directory."
    exit 1
fi

# Switch case to count files
case $directory in
    /*)
        count_files "$directory"
        ;;
    *)
        echo "Invalid directory path."
        exit 1
        ;;
esac
```

```
saad-ali-162@saad-ali-162-VirtualBox:~$ ./lab13_3.sh /usr/bin
Total number of files in '/usr/bin' and its subdirectories: 1283
```

Lab # 13b

1. Implement all the above logic using functions

```
GNU nano 7.2
#!/bin/bash

# Function to check if a number is odd or even
check_odd_even() {
    a=2
    b=0
    echo "Enter the number:"
    read x
    b=$(expr $x % $a)
    if [ $b -eq 0 ]; then
        echo "Given number is Even"
    else
        echo "Given number is Odd"
    fi
}

# Function to reverse a number
reverse_number() {
    echo "Enter the number:"
    read x
    while [ $x -gt 0 ]; do
        c=$(expr $x % 10)
        echo -n "$c"
        x=$(expr $x / 10)
    done
    echo
}

# Function to print the Fibonacci series
print_fibonacci() {
    count=1
    f1=1
    f2=1
    k=0
```

```
    echo $f1
    echo $f2
    while [ $count -le 10 ]; do
        k=$(expr $f1 + $f2)
        echo $k
        f1=$f2
        f2=$k
        count=$(expr $count + 1)
    done
}

# Function to find the average of two numbers
find_average() {
    echo "Enter the 1st number:"
    read a
    echo "Enter the 2nd number:"
    read b
    c=$(expr $a + $b)
    d=$(expr $c / 2)
    echo "The average of two numbers is $d"
}

# Main menu
while true; do
    echo "Select an option:"
    echo "1. Check Odd or Even"
    echo "2. Reverse a Number"
    echo "3. Print Fibonacci Series"
    echo "4. Find Average of Two Numbers"
    echo "5. Exit"
    read choice
    case $choice in
        1)
            check_odd_even
            ;;
        2)
            reverse_number
            ;;
        3)
            print_fibonacci
            ;;
        4)
            find_average
            ;;
        5)
            exit 0
            ;;
        *)
            echo "Invalid choice. Please try again."
            ;;
    esac
done
```

```
saad-ali-162@saad-ali-162-VirtualBox:~$ nano lab13b.sh
saad-ali-162@saad-ali-162-VirtualBox:~$ chmod +x lab13b.sh
saad-ali-162@saad-ali-162-VirtualBox:~$ ./lab13b.sh
Select an option:
1. Check Odd or Even
2. Reverse a Number
3. Print Fibonacci Series
4. Find Average of Two Numbers
5. Exit
1
Enter the number:
4
Given number is Even
```

```
Select an option:
1. Check Odd or Even
2. Reverse a Number
3. Print Fibonacci Series
4. Find Average of Two Numbers
5. Exit
2
Enter the number:
654321
123456
```

```
Select an option:
1. Check Odd or Even
2. Reverse a Number
3. Print Fibonacci Series
4. Find Average of Two Numbers
5. Exit
3
1
1
2
3
5
8
13
21
34
55
89
144
```

```
Select an option:
1. Check Odd or Even
2. Reverse a Number
3. Print Fibonacci Series
4. Find Average of Two Numbers
5. Exit
4
Enter the 1st number:
44
Enter the 2nd number:
332
The average of two numbers is 188
```

Lab # 14

Demonstrating Hello World Example

Pre-requisite

Create an account with [DockerHub](#)

Open [PWD](#) Platform on your browser

Click on Add New Instance on the left side of the screen to bring up Alpine OS instance on the right side

The screenshot displays the Docker PWD Platform interface. On the left sidebar, there is a blue header with a digital clock showing '01:27:41' and a red 'CLOSE SESSION' button. Below this, the 'Instances' section shows a list of instances, with one instance named '192.168.0.13 node1' highlighted. A '+ ADD NEW INSTANCE' button is visible. At the bottom of the sidebar is a 'GIVE FEEDBACK' link. The main panel on the right shows details for the selected instance 'cq2nedii_cq2pklqim2rg00aqh7t0'. It includes an 'IP' address of '192.168.0.13', a 'Memory' usage of '1.27% (50.74MiB / 3.906GiB)', and a 'CPU' usage of '0.70%'. There is an 'OPEN PORT' button. The 'SSH' section shows the command 'ssh ip172-18-0-204-cq2nediim2rg00aqh2vg@direct.labs.pla' with a copy icon. Below this are 'DELETE' and 'EDITOR' buttons. The bottom section shows the output of the 'docker run' command, which is a 'Hello from Docker!' message and a list of steps taken by Docker to generate the message.

```
01:27:41
CLOSE SESSION

Instances

+ ADD NEW INSTANCE

192.168.0.13
node1

GIVE FEEDBACK

cq2nedii_cq2pklqim2rg00aqh7t0

IP
192.168.0.13 OPEN PORT

Memory
1.27% (50.74MiB / 3.906GiB) CPU
0.70%

SSH
ssh ip172-18-0-204-cq2nediim2rg00aqh2vg@direct.labs.pla

DELETE EDITOR

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/
```